

Concept Documentation

Group Members

Charlie Noble-Lepore (a1849667), Rajeswari Roy (a1901014), Lok Hang Li (a1936823), Anna Bratkova (a1961050)

Project Title

The name chosen for our web app is *One Track Mind*. This perfectly encapsulates the unique features of our app: a music-based dating app that only allows users to have one active chat at a time. *One Track Mind* is the perfect place for music lovers to find true connections whilst exploring their own and other's favourite songs, artists, and albums.

Concept Description

The purpose of One Track Mind is to unite adult music lovers and facilitate relationships based on true connection. Users are shown people with a similar music taste, and can swipe to view their top 3 favourite songs, artists, and albums, along with their profile photo and distance. When two users match, they can access further information about the other's music preferences and listening habits. Users can then choose to enter into a live chat with their match to get to know them further. At any point the chat is active, either person can reject the chat to end it, or "complete" the chat and exchange other contact information. Rejected & completed chats are archived and only the match's name and date of last message is visible. While a chat is active, users can access a function to search for artists, songs, or albums, to see how their statistics compare to their matches (e.g. top 5th vs 20th artist). Active chats will also accrue "streaks" and gain badges for achieving certain milestones (e.g. 2 weeks active chat!) which will unlock more functions for the chat users, such as being able to view their match's public playlists or increasing the number of searches available.

At any point, users can access three main pages: one to swipe through other user's profiles, one to access their active and archived chats, and one to view their own profile, including their settings. Users can change their nickname and profile picture, along with their preferences for which users are shown to them (age range, distance, and music tastes priority: would they rather be shown users with similar top artists, songs, or albums?).

API Integration Plan

The Spotify Web API will be used to retrieve information about users' musical preferences, such as their top artists, tracks, and public playlists. When the top items are retrieved using the Spotify API, it returns either an artist or track object.

ArtistObject object	external_urls object Known external URLs for this artist.	genres array of strings A list of the genres the artist is associated with.	images array of ImageObject Images of the artist in various sizes, widest first.	name String The name of the artist.	type String The object type.
TrackObject object	album Object The album on which the track appears. The album object includes a link in href to full information about the album and image of the cover.	artists array of SimplifiedArtistObject The artists who performed the track. Each artist object includes a link in href to more detailed information about the artist.	external_urls Object Known external URLs for this track.	name String The name of the track.	type String The object type.

All retrieved Spotify data will be stored in the user-related tables of the database. This data will be automatically updated every 24 hours using background tasks. This is feasible using Spotify's refresh token system within their OAuth 2.0 flow, which allows long-term access to user data without requiring them to log in again.

The WebSocket API will power a real-time chat system between matched users. When two users are matched, a WebSocket connection is opened between them using the ws library in a Node.js backend. Messages are transmitted in real time and stored in the database. Each message record will include metadata such as sender ID, receiver ID, message content, timestamp, read status, and match ID. If a user "unmatches" with another, all messages linked to that specific match_id are automatically deleted from the database using a simple DELETE query.

Custom Database Plan

One Track Mind will use a MySQL2 object-relational database system to store music preference, chat and user details. Spotify data will be integrated into this relational model through API.

Key Data:

User	Spotify(Using Spotify API)	Match	Chat	Badge
user_id(Primary Key)	user_id(Foreign Key)	match_id(Primary Key)	chat_id(Primary Key)	badge_id(Primary Key)
nickname	top_genres	user_id_1(Foreign Key)	match_id(Foreign Key)	user_id(Foreign Key)
age	top_artists	user_id_2(Foreign Key)	user_id_1(Foreign Key)	date_achieved
location	playlist	match_status	user_id_2(Foreign Key)	badge_name
profile_picture	Top_songs	match_timestamp	chat_status	chat_id(Foreign Key)
active_chatID	last_update		number_of_streaks	
spotify_username			badge	
			message	

This model will integrate one to one relationship between User and Spotify, many to many relationships between User and match and one to many relationship between User and Badge. Normalization of the dataset would ensure efficient storage and reduce redundancy. Scalability is ensured through schema to accommodate future updates while access control would ensure security and reliability.

User Service Description

Our application aims to deliver a unique experience by building true connections through shared music interests. By getting user’s listening habits from Spotify, the app can provide personalised suggestions to match with similar music tastes. This approach enhances the chance to find matches that users can truly connect with. In addition, users can only establish connections through mutual matches, fostering a safe and intentionally driven environment. Both users must match each other before starting to chat, intending to eliminate the discomfort of one-side interactions, facilitating genuine connections. To further achieve authentic interaction, the filter function allows users to exclude unwanted condition matches, for example age or distances, helping them to find their “perfect match”. By utilising Spotify API, our app offers music driven conversation prompts based on users’ personal music taste, from their top 50 songs, artists, and more, making conversations more engaging and natural through shared interests.

Team Collaboration Plan

To ensure effective teamwork and smooth progress, our team has established a clear and structured approach. Each member is assigned specific tasks with defined due dates to ensure a fair and reasonable workload. Roy and Anna will be focusing on implementing Database. Charlie will be responsible for developing Backend. The Frontend will be developed by Roy, Megan, and Charlie. And Anna will take control of the Spotify API. The WebSockets API will be developed jointly by Charlie and Roy. Lastly, all team members will participate in final Testing.

Our team has server strategies to communicate. We use WhatsApp for day-to-day communication, including organising group meeting scheduling and discussing any issues or challenges that arise during the assignment. Regular meetings are scheduled every Tuesday and Friday to make sure the progress is on track. To manage our workflow organized, we utilize Google Docs, GitHub, and Figma. Google Docs is used for collaborating on written work, creating to-do list and action plans, helps to track completed tasks, identify undone work, and delegating responsibilities among team members. To support technical collaboration, we have shared a GitHub repository for storing coding work and other related files. Finally, we use Figma for collaborative front-end design, enabling real-time cooperation on UI/UX development.

Development Timeline

Date to complete by	Task name	Task description	Delegated to
2nd May	Database mySQL2		
18th April	Develop relationship schema	Specify requirements of the database and develop a relationship schema. Group related data, remove unnecessary data. Select/add primary keys, build relationships, determine relationship cardinality. Improve Schema through normalisation and normal forms. Use an ER diagram to plan visually in https://erdplus.com/ .	Roy & Megan
25th April	Implement relationship schema	Use mySQL2 (driver for NodeJS) to start a database management system, create databases and tables within databases.	Roy & Anna

2nd May	Connect to application	Use MySQL2 to send queries from the backend. Start with simple queries, e.g. SELECT * FROM users.	Anna
2nd May	Implement user roles & permissions	Restrict actions for normal users vs admin, ensure privileges/permissions reflect project requirements.	Anna
9th May	Backend (Server-side) Node.js, Express, AJAX, JS		
18th April	Node.js	Initialise Node.js project, setup package.json, install Express, and structure folders.	Charlie
18th April	Express	Implement a basic Express app that listens on a port.	Charlie
2nd May	Routes	Implement routes for core features, such as endpoints:/register, /login, /profile, /matches, /chat. Break this into User, Matching, and Chat routes.	Charlie & Megan
2nd May	Middleware functions	Set up middleware functions for authentication, validation, error handling. Implement JWT-based auth & validate user input. Use express-validator for form validation.	Charlie & Megan
9th May	Connect to MySQL2	Implement queries for GET /profile, POST /match, DELETE /chat.	Charlie & Anna
9th May	Error handling	Centralized error handling for API/database errors. Use express-async-errors and logging tools.	Anna
16th May	API Spotify API		
25th April	Implement OAuth 2.0	Allow users to login via Spotify, store tokens. Use passport-spotify for authentication.	Anna
2nd May	Fetch Spotify data	Get user's top songs, genres, artists, and public playlists. Use /me/top/artists and /me/top/tracks.	Anna
9th May	Store API data	Store relevant Spotify data in MySQL. Auto-update every 24 hours. Use a cron job (e.g., node-cron).	Roy & Anna
16th May	Rate limit	Rate limit API calls to prevent exceeding Spotify's rate limits. Use caching (e.g.store in DB).	Roy
9th May	Frontend (Client-side) HTML, CSS, Javascript, Vue		
18th April	Design of pages and content	Brainstorm layout/content/links of pages with drawings and consider responsive design features for user interactions. Prioritise user experience by minimising performance load and maintaining accessibility in site design. Set up Figma for collaborative design and development.	Charlie & Anna
25th April	HTML	Implement structure and content of pages, including login, profile, match, and chat pages. Keep accessibility in mind (aria-labels). Implement semantic HTML where reasonable and metadata to add a description & keywords for the site.	Charlie & Megan
25th April	CSS	Implement a CSS framework as the basis for design. Set up layout, design, and responsiveness of pages. Review IXD checklist for accessibility and user experience.	Megan & Roy
2nd May	Vue	Initialise Vue project and set up components. Use Vue Router for navigation.	Megan & Roy
2nd May	AJAX	Implement AJAX requests. Fetch data from backend API (e.g. /matches). Potentially use Axios to send HTTP requests.	Charlie
9th May	Handle dynamic updates	Live update matches, chat messages, etc. use VUE state management (Pinia)	Megan
16th May	API WebSockets API		
9th May	Server	Set up WebSocket server, allowing real-time chat using socket.io. Implement io.on("message", ...) to handle messages.	Charlie & Roy
9th May	Connect to MySQL2	Implement message history by storing chat logs in MySQL. Design chats table with timestamps.	Charlie & Roy
16th May	Limit to one active chat per user	Ensure users can't chat with multiple people at once by using a status field in the matches table.	Charlie
6th June	Testing & final adjustments Test cases and optimisations, security		
30th May	Backend unit tests	Use Jest to write unit tests to test backend authentication, database queries, and routes.	Charlie
30th May	Frontend validation	Ensure forms reject invalid input using VeeValidate in Vue.	Roy
30th May	Integration tests	Test API endpoints and database together using Postman.	Anna
30th May	End-to-end tests	Ensure each feature functions according to requirements, check for vulnerabilities, and check the application across different browsers and devices. If time allows, use Cypress to simulate full user flow, i.e. register/login, match, chat, finish chat, new match, etc.	Megan
6th June	Optimise queries	Ensure database queries are efficient, use indexes and avoid SELECT *, optimize with proper indexing.	Anna
6th June	Secure application	Implement session timeout, HTTPS, JWT expiration. Download and use the Helmet npm package.	Charlie & Roy
6th June	Final checks	Use W3C Validation for HTML & CSS files. Install & configure ESLint to check Javascript code.	Megan

